

Department of Computer Science and Engineering

Theory Quiz - 3 [S-26]

CSE220: Data Structure

Duration: 45 min

Name:	ID:	Section:
-------	-----	----------

Question 1 [5]

1. Which traversal always visits the keys of a BST in ascending sorted order?

a) Pre-order b) In-order c) Post-order d) Level-order

2. Which of the following correctly defines a **Full Binary Tree**?

a) Every node has either exactly 2 children or 0 children (leaf)

b) Every level is completely filled

c) Every leaf is at the same depth

d) Every node must have at least 1 child

3. Keys 50, 30, 70, 20, 40, 60, 80, 10 are inserted into a BST in this order. How many comparisons are needed to reach key = 10 starting from the root? (a single node has height 0)

a) 2 b) 3 c) 4 d) 5

4. When deleting a BST node that has two children, it is typically replaced by:

a) Its parent

b) Its in-order successor or in-order predecessor

c) The root node

d) Any random leaf node

5. In a Full Binary Tree, if the number of **internal nodes** (nodes with 2 children) is i , how many **leaf nodes** does the tree have?

a) i

b) $i + 1$

c) $2i$

d) $i - 1$

Question 2 [6]

You are given the root of a BST, along with two integers, low and high. Write a function `rangeSumBST()` that returns the sum of all node values in the BST that lie between low and high(inclusive).

No extra data structures are allowed.

Assume the Node class has: `(int) val, (Node) left, (Node) right`

Example:

BST:

```
      50
     /  \
    30   70
   / \  / \
  20 40 60 80
```

`rangeSumBST(root, 30, 65)`

→ $30 + 40 + 50 + 60 = 180$

Python:

```
def rangeSumBST(root, low, high):
    # TODO
```

Java:

```
static int rangeSumBST(Node root, int
low, int high) {

    // TODO

}
```

Question 3 [4]

Insert the following keys into an empty BST, in this order: 45, 25, 65, 15, 35, 55, 75, 5, 20

a) Show, step by step, how the tree looks after each key is inserted (or clearly draw the final resulting tree).

b) Now delete the node 25 (which has two children) using its **in-order successor**, and show the tree after the deletion.

c) What is the height of the final tree? (assume a single node has height 0)